

অবশ্যই। Module 2 — Forward & Backpropagation Neural Network-এর সবচেয়ে গুরুত্বপূর্ণ অংশগুলোর একটি। এখানে মূল বিষয় হলো:

Model কীভাবে prediction করে → ভুল বের করে → ভুলের কারণ খুঁজে → weights পরিবর্তন করে → আবার ভালো prediction করে।

একটা simple example দিয়ে পুরো workflow-টা বুঝি।

Module 2 — Forward & Backpropagation

ধরো আমরা একটি model বানাচ্ছি:

Student pass করবে কি না predict করবে।

Input:

- x_1 = Study Hours
- x_2 = Attendance

Target:

- Pass = 1
 - Fail = 0
-

1. Forward Propagation ★★ ★

সংজ্ঞা

Forward Propagation হলো input data-কে Neural Network-এর মধ্য দিয়ে সামনে পাঠিয়ে final prediction বের করার process।

Workflow:

```
Input
↓
Weight দিয়ে multiply
↓
Bias যোগ
↓
Activation
↓
Hidden Layer
↓
Output Layer
↓
Prediction
```

Example

ধরো:

```
Study = 5
Attendance = 80
```

Network এগুলো নিয়ে calculation করবে:

$$z = w_1x_1 + w_2x_2 + b$$

তারপর:

$$a = \text{Activation}(z)$$

শেষে model বলবে:

```
Prediction = 0.8
```

অর্থাৎ model মনে করছে:

Pass করার probability $\approx$ 80%



2. Loss Function ★★ ★

সংজ্ঞা

Loss Function বলে দেয় model-এর prediction কতটা ভুল হয়েছে।

অর্থাৎ:

Prediction এবং Actual Answer-এর মধ্যে কতটা difference আছে তা measure করে Loss।

ধরো:

Actual = 1
Prediction = 0.8

তাহলে কিছু loss হবে।

আবার:

Actual = 1
Prediction = 0.1

এখানে ভুল অনেক বেশি, তাই loss-ও বেশি হবে।

সহজভাবে

Prediction কাছাকাছি Actual
↓
Low Loss ✓

Prediction অনেক দূরে Actual
↓
High Loss ✗

Common Loss Functions

MSE — Mean Squared Error

Regression-এর জন্য খুব common:

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

Example:

Actual = 10
Prediction = 8

Error = $10 - 8 = 2$

Squared Error = 4

Cross-Entropy Loss

Classification-এর জন্য খুব গুরুত্বপূর্ণ।

যেমন:

Cat / Dog
Spam / Not Spam
Pass / Fail

বিশেষ করে neural network classification-এ এটি খুব commonly ব্যবহৃত হয়।

3. Gradient ★★ ★

এটা প্রথমে একটু কঠিন মনে হতে পারে।

সংজ্ঞা

Gradient বলে দেয় কোন direction-এ এবং কতটা পরিবর্তন করলে Loss কমবে।

সহজভাবে:

Gradient = "আমার ভুল কোন দিকে এবং কতটা?"

ধরো তুমি পাহাড়ের উপরে আছো এবং নিচে নামতে চাও।



Gradient তোমাকে বলে:

কোন দিকে গেলে loss কমবে।

4. Gradient Descent ★★ ★

Gradient জানলাম।

এখন Gradient ব্যবহার করে Loss কমানোর জন্য weights পরিবর্তন করার algorithm হলো Gradient Descent।

মূল ধারণা

```
High Loss
↓
Calculate Gradient
↓
Move opposite direction
↓
Lower Loss
↓
Repeat
```

Weight update:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

এখানে:

- w = weight
- L = loss
- η = learning rate

Example

ধরে:

```
Old weight = 0.5
Gradient = 0.2
Learning rate = 0.1
```

তাহলে:

$$\begin{aligned} w_{new} &= 0.5 - (0.1)(0.2) \\ &= 0.48 \end{aligned}$$

অর্থাৎ weight:

```
0.50 → 0.48
```

5. Learning Rate ★★ ★

সংজ্ঞা

Learning Rate নির্ধারণ করে প্রতিবার weight কত বড় step-এ পরিবর্তন হবে।

অর্থাৎ:

Learning Rate = শেখার step size

ছোট learning rate

0.001



Model খুব ছোট ছোট step নেয়।

✓ Stable হতে পারে

✗ Training অনেক slow হতে পারে

বড় learning rate

0.1



বড় step নেয়।

✓ দ্রুত এগোতে পারে

✗ Loss-এর minimum-এর চারপাশে লাফাতে পারে বা training unstable হতে পারে।

সহজ analogy:

Learning rate ছোট:

• → • → • → • → Minimum



Learning rate বড়:

• → ←

Minimum-এর উপর দিয়ে চলে যেতে পারে

6. Chain Rule ★★ ★

Backpropagation বুঝতে Chain Rule খুব important!

সংজ্ঞা

Chain Rule ব্যবহার করে বোঝা যায়:

একটি weight পরিবর্তন হলে final loss কতটা পরিবর্তিত হবে।

Neural network-এ অনেকগুলো function একটার পর একটা থাকে:

```
Input
↓
z
↓
Activation
↓
Prediction
↓
Loss
```

তাই loss-এর ওপর একটি weight-এর প্রভাব বের করতে একাধিক derivative একসাথে multiply করতে হয়।

উদাহরণ:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial z} \times \frac{\partial z}{\partial w}$$

সহজ analogy

ধরে:

```
A → B → C → D
```

A-এর পরিবর্তন D-কে কতটা affect করছে জানতে হলে প্রতিটি step-এর effect combine করতে হবে।

এটাই Chain Rule-এর মূল ধারণা।

7. Backpropagation ★★ ★★ ★

এটাই Module 2-এর সবচেয়ে important concept!

সংজ্ঞা

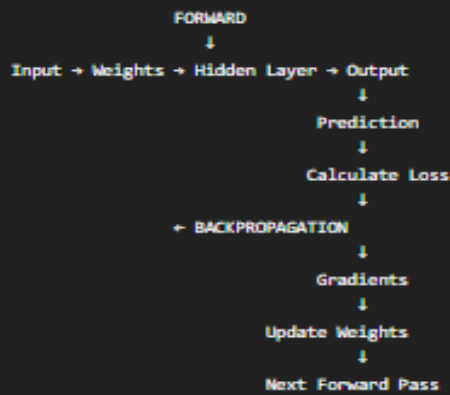
Backpropagation হলো prediction-এর error বা loss-কে network-এর output থেকে পিছনের দিকে পাঠিয়ে প্রতিটি weight-এর gradient বের করার process।

Workflow:

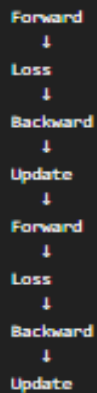
```
Input
↓
Forward Propagation
↓
Prediction
↓
Loss
↓
Backpropagation
↓
Gradient
↓
Update Weights
```

🔥 Forward + Backpropagation একসাথে

এটাই সবচেয়ে important workflow:



তারপর আবার:



এই process বারবার চলে।

8. Epoch ★★

সংজ্ঞা

একটি সম্পূর্ণ training dataset model-এর মধ্য দিয়ে একবার পুরোপুরি যাওয়াকে 1 epoch বলে।

ধরো তোমার কাছে:

1000 students' data

Model যদি পুরো 1000 data একবার process করে:

- 1 Epoch

দুইবার:

- 2 Epochs

9. Batch ★★

সংজ্ঞা

Training data-কে একসাথে যে সংখ্যক sample নিয়ে model process করে সেটি batch।

ধরো:

```
Dataset = 1000 samples  
Batch size = 1000
```

তাহলে পুরো dataset একসাথে process হবে।

```
1000 → Forward → Loss → Backprop → Update
```

এটা Full Batch Gradient Descent।

10. Mini-Batch ★★ ★

বড় dataset একবারে পুরোটা process না করে ছোট ছোট group-এ ভাগ করলে তাকে Mini-Batch বলা হয়।

ধরো:

```
Dataset = 1000 samples  
Batch size = 100
```

তাহলে:

```
Batch 1 → 100  
Batch 2 → 100  
Batch 3 → 100  
...  
Batch 10 → 100
```

প্রতিটি batch-এর পরে weights update হতে পারে।

```
Batch  
↓  
Forward  
↓  
Loss  
↓  
Backprop  
↓  
Update Weights  
↓  
Next Batch
```

কেন Mini-Batch popular?

কারণ এটি:

- Full dataset-এর চেয়ে কম memory ব্যবহার করতে পারে
- Pure single-sample training-এর তুলনায় efficient
- Modern deep learning-এ খুব commonly ব্যবহৃত

🔥 Epoch vs Batch — খুব সহজে

ধরো:

```
Dataset = 1000 students  
Batch size = 100
```



একটি epoch-এ:

```
1000 / 100 = 10 batches
```



অর্থাৎ:

```
Epoch 1  
  
Batch 1 → Train  
Batch 2 → Train  
Batch 3 → Train  
...  
Batch 10 → Train  
  
= 1 Epoch Complete
```



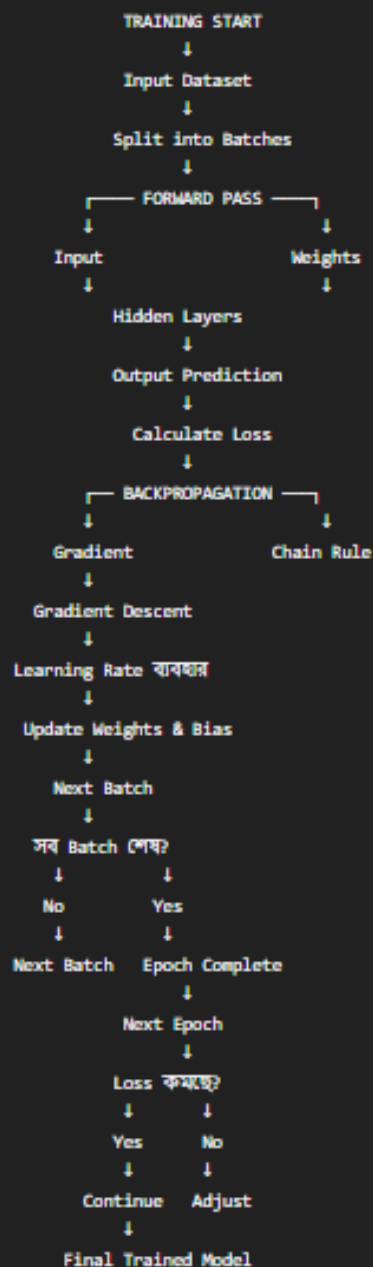
তারপর আবার:

```
Epoch 2  
Batch 1 → Train  
...  
Batch 10 → Train
```



✿ পুরো Neural Network Training Workflow

এখন সব concept একসাথে বসাও:



একটি Real Numerical Example

ধরো:

Input:

$x = 2$

Weight:

$w = 0.5$

Bias:

$b = 0.1$

Step 1 — Forward

$$z = wx + b$$

$$z = (0.5)(2) + 0.1$$

$$z = 1.1$$

ধরো activation-এর পরে:

Prediction = 0.75

Actual:

$y = 1$

Step 2 — Loss

Model বলেছে:

0.75

কিন্তু actual:

1

তাই কিছু loss হয়েছে।

Step 3 — Gradient

Backpropagation + Chain Rule দিয়ে আমরা বের করি:

Weight-এর জন্য gradient কত?
Bias-এর জন্য gradient কত?



ধরো gradient পাওয়া গেল:

Gradient = 0.3



Step 4 — Weight Update

Learning rate:

$\eta = 0.1$



পুরনো weight:

0.5



তাহলে:

$$w_{new} = 0.5 - (0.1)(0.3)$$

$$w_{new} = 0.47$$

অর্থাৎ:

Old Weight = 0.50
New Weight = 0.47



Step 5 — আবার Forward

Updated weight দিয়ে আবার:

Input
↓
New Weight
↓
Forward Pass
↓
New Prediction
↓
New Loss



যদি loss কমে:

Old Loss = 0.30
New Loss = 0.20



তাহলে model better হচ্ছে।



